SSX-32: Instruction Set Architecture Manual

36-Instruction Catalog, Opcode Encodings, and Bitfield Mappings

Author: Samin Sami (SS) **Architecture:** SSX-32 (32-Bit Dual Engine) **Status:** Formally Verified (100% Pass)

SSX-32 instruction set reference manual

Creator: Samin Sami (SS)
Architecture: SSX Silicon 1

Instruction format

SSX Silicon 1 uses a fixed 32-bit primary instruction word format. Instructions that require a full 32-bit immediate constant or an absolute 32-bit branch target take two consecutive 32-bit words in memory.

Primary instruction word layout (32 bits)

```
 31       24 23     19 18     14 13      9 8              0
+-----------+---------+---------+---------+-----------------+
| OPCODE[7:0] | RD[4:0] | RS1[4:0]| RS2[4:0]| IMM9 / SUB-FUNC |
+-----------+---------+---------+---------+-----------------+
```

- Bits [31:24] (8 bits): Opcode (0x00 to 0xFF).
- Bits [23:19] (5 bits): Destination register Rd (0 to 31).
- Bits [18:14] (5 bits): First source register Rs1 (0 to 31).
- Bits [13:9] (5 bits): Second source register Rs2 (0 to 31).
- Bits [8:0] (9 bits): 9-bit signed immediate value (-256 to +255), shift count (0 to 31), or sub-function code.

Two-word instructions (64 bits total)

```
Word 0: [OPCODE: 8 bits][RD: 5 bits][RS1: 5 bits][UNUSED: 14 bits]
Word 1: [32-bit Immediate Value / Absolute Target Address]
```

Instructions that use the two-word format include LOADI, STOREI, JMP, CALL, and conditional jumps.

Opcode reference table

Opcode	Mnemonic	Operands	Operation description	Flags modified
0x00	NOP		No operation	None
0x01	HLT		Halts instruction execution	None
0x02	RESET		Resets all registers and sets PC to 0	None
0x03	SYS	#code	Triggers a system call using code in immediate field	None
0x04	CLC		Clears Carry flag (CF = 0)	CF
0x05	STC		Sets Carry flag (CF = 1)	CF
0x10	MOV	Rd, Rs	Rd = Rs	None
0x11	LOAD	Rd, [Rs]	Rd = Memory[Rs]	None
0x12	STORE	Rs, [Rd]	Memory[Rd] = Rs	None
0x13	LOADI	Rd, #imm32	Rd = imm32 (two words)	None
0x14	STOREI	#imm32, [Rd]	Memory[Rd] = imm32 (two words)	None
0x15	PUSH	Rs	SP = SP - 1; Memory[SP] = Rs	None
0x16	POP	Rd	Rd = Memory[SP]; SP = SP + 1	None
0x20	ADD	Rd, Rs1, Rs2	Rd = Rs1 + Rs2	ZF, CF, SF, OF
0x21	ADDI	Rd, Rs1, #imm9	Rd = Rs1 + sign_extend(imm9)	ZF, CF, SF, OF
0x22	SUB	Rd, Rs1, Rs2	Rd = Rs1 - Rs2	ZF, CF, SF, OF
0x23	SUBI	Rd, Rs1, #imm9	Rd = Rs1 - sign_extend(imm9)	ZF, CF, SF, OF
0x24	MUL	Rd, Rs1, Rs2	Rd = (Rs1 *Rs2)[31:0]; R27 = (Rs1* Rs2)[63:32]	ZF, SF
0x25	DIV	Rd, Rs1, Rs2	Rd = Rs1 / Rs2; R27 = Rs1 % Rs2	ZF, SF
0x26	MOD	Rd, Rs1, Rs2	Rd = Rs1 % Rs2	ZF, SF
0x27	INC	Rd	Rd = Rd + 1	ZF, SF
0x28	DEC	Rd	Rd = Rd - 1	ZF, SF

Opcode	Mnemonic	Operands	Operation description	Flags modified
0x29	NEG	Rd	Rd = -Rd	ZF, SF
0x30	AND	Rd, Rs1, Rs2	Rd = Rs1 & Rs2	ZF, SF
0x31	ANDI	Rd, Rs1, #imm9	Rd = Rs1 & imm9	ZF, SF
0x32	OR	Rd, Rs1, Rs2	Rd = Rs1 | Rs2	ZF, SF
0x33	ORI	Rd, Rs1, #imm9	Rd = Rs1 | imm9	ZF, SF
0x34	XOR	Rd, Rs1, Rs2	Rd = Rs1 ^ Rs2	ZF, SF
0x35	XORI	Rd, Rs1, #imm9	Rd = Rs1 ^ imm9	ZF, SF
0x36	NOT	Rd	Rd = ~Rd	ZF, SF
0x37	SHL	Rd, Rs, #shift	Logical left shift by shift count (0 to 31)	ZF, SF
0x38	SHR	Rd, Rs, #shift	Logical right shift (zero-fill)	ZF, SF
0x39	SAR	Rd, Rs, #shift	Arithmetic right shift (sign-fill)	ZF, SF
0x3A	ROL	Rd, Rs, #shift	Rotate left by shift count	ZF, SF
0x3B	ROR	Rd, Rs, #shift	Rotate right by shift count	ZF, SF
0x40	CMP	Rs1, Rs2	Evaluates Rs1 - Rs2 to set condition flags	ZF, CF, SF, OF
0x41	CMPI	Rs1, #imm9	Evaluates Rs1 - sign_extend(imm9) to set flags	ZF, CF, SF, OF
0x42	TST	Rs1, Rs2	Evaluates Rs1 & Rs2 to set condition flags	ZF, SF
0x50	JMP	#target	PC = target address	None
0x51	JE / JZ	#target	Jump if ZF = 1 (equal / zero)	None
0x52	JNE / JNZ	#target	Jump if ZF = 0 (not equal / non-zero)	None
0x53	JL	#target	Jump if SF != OF (signed less than)	None
0x54	JLE	#target	Jump if ZF = 1 or SF != OF (signed less than or equal)	None
0x55	JG	#target	Jump if ZF = 0 and SF == OF (signed greater than)	None
0x56	JGE	#target	Jump if SF == OF (signed greater than or equal)	None

Opcode	Mnemonic	Operands	Operation description	Flags modified
0x57	JC	#target	Jump if CF = 1 (carry set / unsigned lower)	None
0x58	JNC	#target	Jump if CF = 0 (carry clear / unsigned higher or equal)	None
0x59	CALL	#target	Pushes return address to stack, sets LR, jumps to target	None
0x5A	RET		Pops return address from stack to PC	None
0x60	OUT	#port, Rs	Writes value of Rs to hardware I/O port	None
0x61	IN	Rd, #port	Reads hardware I/O port into Rd	None
0x62	VSET	Rx, Ry, Rc	Sets pixel at coordinate (Rx, Ry) to 4-bit color in Rc	None
0x63	VCLR	#color	Clears entire 32x32 display to specified color	None

Instruction details and examples

Data movement

LOADI: Load 32-bit immediate constant

Loads a 32-bit literal into the destination register. Takes two 32-bit words.

```
LOADI R1, 0x12345678  ; R1 = 0x12345678
LOADI R2, 1000        ; R2 = 1000 decimal
```

LOAD: Load word from memory

Reads a 32-bit word from the memory address specified in the source register.

```
LOADI R10, 0x1000     ; R10 holds base address
LOAD  R1, [R10]       ; R1 = Memory[0x1000]
```

STORE: Store word to memory

Writes a 32-bit word to the memory address specified in the target register.

```
LOADI R10, 0x1000     ; R10 holds base address
LOADI R1, 0xCAFE      ; Data value
STORE R1, [R10]       ; Memory[0x1000] = 0xCAFE
```

Arithmetic and logic

ADD and ADDI: Addition

Performs 32-bit addition with full flag generation.

```
ADD   R3, R1, R2      ; R3 = R1 + R2
ADDI  R1, R1, 1       ; R1 = R1 + 1 (using 9-bit immediate)
```

MUL: 32-bit integer multiplication

Multiplies two 32-bit values. The lower 32 bits of the 64-bit product are stored in Rd. The upper 32 bits are written to R27.

```
LOADI R1, 65536
LOADI R2, 131072
MUL   R3, R1, R2      ; R3 = lower 32 bits (0x00000000)
                      ; R27 = upper 32 bits (0x00000002)
```

DIV: 32-bit integer division

Divides Rs1 by Rs2. The quotient is stored in Rd. The remainder is written to R27.

```
LOADI R1, 100
LOADI R2, 7
DIV   R3, R1, R2      ; R3 = 14 (quotient)
                      ; R27 = 2 (remainder)
```

Branching and subroutines

JMP and conditional jumps

Evaluates condition codes in the status register and branches to an absolute 32-bit address.

```
CMP   R1, R2          ; Compare R1 and R2
JE    TARGET_EQUAL    ; Branch if R1 == R2 (ZF = 1)
JL    TARGET_LESS     ; Branch if R1 < R2 (signed)
JMP   TARGET_OTHER    ; Unconditional branch
```

CALL and RET: Subroutine linkage

CALL pushes the address of the next instruction onto the memory stack (decrements SP, writes return address to Memory[SP]), copies that address to the link register R30 (LR), and jumps to the subroutine target. RET pops the return address from the stack and restores it to PC.

```
LOADI R1, 5
CALL  DOUBLE_VAL      ; Call subroutine
; Execution returns here with R1 = 10
HLT

DOUBLE_VAL:
ADD   R1, R1, R1      ; R1 = R1 * 2
RET                   ; Return to caller
```